SYRIAN ARAB REPUBLIC

Higher Institute for Applied Sciences and Technology



NETWORK COMMUNICATION

Transferring Complex Data Objects in Distributed Systems

A Comparative Study of JSON, Java Object Serialization, and Protocol Buffers

PREPARED BY	**Eng. Mohammad Ghaith al-Atrash**
LABORATORY	Network Communication Laboratory
TOPIC	Data Serialization and Deserialization in Distributed Systems
DATE	7 November 2026

Abstract

Modern distributed systems are built from independent processes that run on different machines, in different programming languages, and under the responsibility of different teams. These processes can cooperate only if they can exchange structured data, and exchanging structured data means converting in-memory objects into a flat sequence of bytes or characters (*serialization*) and rebuilding equivalent objects at the other end (*deserialization*).

This report documents a three-part laboratory investigation of that mechanism, organized around one running scenario: the transfer of a `CustomerPurchaseProfile` object between the nodes of an online store's analytics system. **Experiment 1** represents the object in JSON, sends it over HTTP, and reproduces a type mismatch between two services in order to show what happens when no schema defines the data contract. **Experiment 2** applies Java's built-in object serialization, examines how `transient` and `static` members are treated, provokes a `NotSerializableException`, and inspects the binary payload. **Experiment 3** describes the same object in a Protocol Buffers schema, generates code from it, measures the resulting message, and tests schema evolution by renaming a field.

For identical data the measurements show a clear ranking: **35 bytes** for Protocol Buffers, **118 bytes** for compact JSON, and **322 bytes** for standard Java serialization. The Protocol Buffers message is therefore about 70% smaller than the JSON message and about 89% smaller than the Java-serialized one. The experiments also show why size is only part of the picture. JSON is the most interoperable and the easiest to debug, but it offers no contract; Java serialization is the most convenient inside a homogeneous Java environment, but it is bound to the JVM and exposes class metadata; Protocol Buffers combine compactness with an enforced, language-neutral schema. The report closes with selection guidelines for four representative system designs.

Keywords: serialization, deserialization, JSON, Java object serialization, Protocol Buffers, distributed systems, data schema, interoperability.

Table of Contents

1. Introduction 1
1.1 Background and Motivation 1
1.2 Objectives of the Study 1
1.3 Structure of the Report 1

2. Theoretical Background 2
2.1 Serialization and Deserialization 2
2.2 The Three Techniques 2
2.3 Comparative Overview 3

3. Application Scenario and Experimental Environment 4
3.1 The CustomerPurchaseProfile Scenario 4
3.2 Experimental Environment 5

4. Experiment 1: JSON Serialization and the Lack of a Schema 6
4.1 Objective and Rationale 6
4.2 Task 1.1: Building and Testing the JSON Object 6
4.3 Task 1.2: Simulating the Absence of a Single Source of Truth 8
4.4 Report Question 1 9
4.5 Summary of Findings 9

5. Experiment 2: Standard Java Serialization 10
5.1 Objective and Rationale 10
5.2 Task 2.1: Writing the Serializable Class 10
5.3 Task 2.2: Performing Serialization and Deserialization 11
5.4 Task 2.3: Results and Observations 12
5.5 Summary of Findings 16

6. Experiment 3: Google Protocol Buffers 17
6.1 Objective and Rationale 17
6.2 Preparing the Environment 17
6.3 Task 3.1: Defining the Schema and Generating the Stub 17
6.4 Task 3.2: Building the Object, Serializing It, and Measuring the Result 18
6.5 Report Question 2 (a): Comparing the Message Sizes 21
6.6 Report Question 2 (b): Renaming a Field on the Receiver Only 22
6.7 Summary of Findings 24

7. Comparative Analysis of the Three Techniques 25
7.1 Consolidated Experimental Results 25
7.2 Interpretation 26

8. Design Decisions: Choosing a Serialization Technique 27
8.1 Case 1: Shopping-Cart Data from a Browser to the Back End 27
8.2 Case 2: A Fast-Moving Java-Only Prototype 28
8.3 Case 3: A Large Multi-Language Trading System 28
8.4 Additional Question: Why a Deleted Field's Tag Must Not Be Reused 29

9. Conclusion 30
9.1 Summary of Results 30
9.2 Recommendations 30
Appendix A: Practical Notes and Troubleshooting 31
A.1 Quoting JSON in the Windows Command Prompt 31
A.2 JVM Warnings Printed by the Protobuf Runtime 31
A.3 Keeping protoc and the Runtime Library Aligned 31
A.4 Making Binary Payloads Easier to Read 31
References 32

List of Figures

Figure 1. The serialization and deserialization pipeline between two nodes. 2
Figure 2. Successful request: the server returns the customer name and the total price (1310). 7
Figure 3. Type-mismatch failure: the receiver raises a TypeError when prices arrive as strings. 8
Figure 4. The program fails with java.io.NotSerializableException: Address. 14
Figure 5. Output of ProtobufLab: size, restored values, and the default city. 19
Figure 6. Byte-level layout of the 35-byte Protocol Buffers payload. 20
Figure 7. The receiver parses the unchanged payload using the renamed field. 23
Figure 8. Serialized size of identical data: Protocol Buffers, compact JSON, and Java serialization. 25

List of Tables

Table 1. Reference comparison of the three serialization techniques. 3
Table 2. Attributes of the CustomerPurchaseProfile object. 4
Table 3. Software environment used for the experiments. 5
Table 4. The two incompatible representations of the prices field. 8
Table 5. Elements of the CustomerProfile class and their effect on serialization. 11
Table 6. Field values before serialization and after deserialization. 13
Table 7. Readable fragments found in the Java-serialized payload. 15
Table 8. Fields of the CustomerProfile message. 18
Table 9. Decomposition of the 35-byte payload. 20
Table 10. Serialized size of identical data in each format. 21
Table 11. Evidence gathered in the laboratory for each technique. 25
Table 12. Summary of recommended techniques. 27

1. Introduction

1.1 Background and Motivation

In a distributed system, components communicate by sending messages across a network. A network link, however, can carry only bytes; it knows nothing about the objects, lists, and records that a program manipulates in memory. Before an object can leave one process, it must therefore be transformed into a transportable representation, and the receiving process must be able to turn that representation back into an equivalent object. These two operations are called **serialization** and **deserialization** (also known as marshalling and unmarshalling).

The choice of serialization technique is not a minor implementation detail. It determines how large each message is, how quickly it can be produced and parsed, whether components written in different languages can interoperate, how easily engineers can inspect traffic when something fails, how much of a system's internal structure is exposed to anyone who intercepts a message and, most importantly for long-lived systems, how safely the data format can change over time. A format that is convenient for a prototype can become a liability in production, and the reverse is also true.

This report examines three widely used options (JSON, standard Java object serialization, and Google Protocol Buffers) through hands-on experiments, and uses the observed results to explain when each one is the appropriate engineering choice.

1.2 Objectives of the Study

The laboratory was designed so that, on completion, the following objectives are met:

- Understand the concepts of serialization and deserialization and their role in moving complex objects across a network using the HTTP protocol.
- Represent a complex object in JSON and analyze the problem caused by the absence of a unified data schema.
- Apply standard Java serialization through the `Serializable` interface, and handle `transient` and `static` fields correctly.
- Write a Protocol Buffers schema (`.proto`), generate the corresponding classes (stubs), and compare message size, safety, and efficiency with the other approaches.

1.3 Structure of the Report

Section 2 summarizes the theoretical background and gives a reference comparison of the three techniques. Section 3 introduces the application scenario and the experimental environment. Sections 4, 5, and 6 document Experiments 1 to 3; each presents the objective, the procedure, the source code, the observed results, and an analysis. Section 7 consolidates the measurements into a comparative evaluation, Section 8 answers the closing design questions, and Section 9 concludes. Appendix A records practical issues that arose during the work and how they were resolved.

2. Theoretical Background

2.1 Serialization and Deserialization

Serialization is the process by which the sender converts an in-memory object into a textual form or a stream of bytes that can be transmitted across a network or stored. **Deserialization** is the reverse process, in which the receiver rebuilds the original object in memory from the data it received. Figure 1 shows the complete path from the sender's object to the receiver's object.

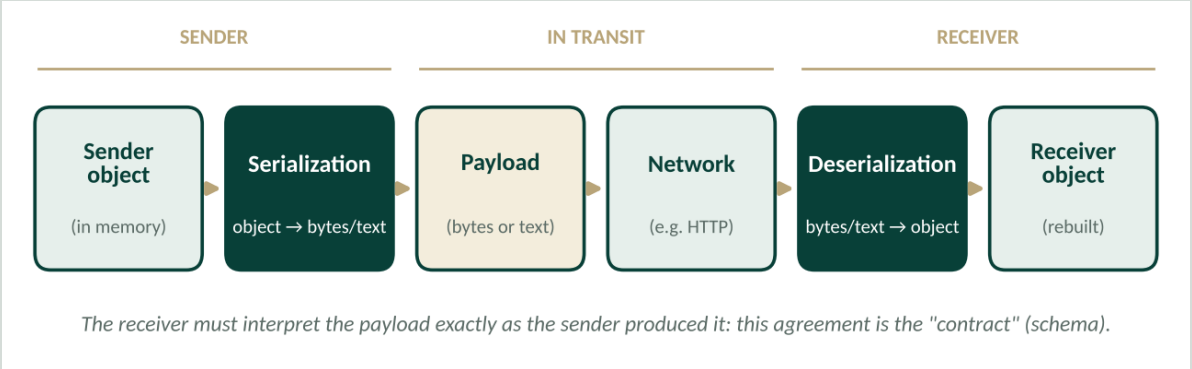


Figure 1. The serialization and deserialization pipeline between two nodes.

For the exchange to succeed, both sides must agree on the representation: which fields exist, what their types are, how they are identified, and how they are encoded. This agreement is the **data contract**. The three techniques studied in this laboratory differ mainly in how that contract is expressed and enforced: not at all in JSON, implicitly by a Java class in Java serialization, and explicitly by a schema file in Protocol Buffers.

2.2 The Three Techniques

JSON (JavaScript Object Notation)

JSON [1] is a plain-text format built from objects (key–value pairs), arrays, strings, numbers, booleans, and `null`. It is supported natively by browsers and by libraries in virtually every programming language, and it is directly readable by humans, which makes it easy to inspect, edit, and debug. Its weaknesses follow from the same properties. Every field name is repeated as text in every message, so messages are comparatively large and slower to parse. In addition, the format carries no declaration of the type each field is supposed to have: the string `"1200"` and the number `1200` are both valid JSON, yet they are not interchangeable.

Java Object Serialization

Standard Java serialization [2] converts an object graph into a binary byte stream through `ObjectOutputStream` and restores it through `ObjectInputStream`. A class opts in by implementing the empty marker interface `java.io.Serializable`. No separate schema is needed, because the class definition itself acts as the schema, which makes the technique extremely convenient. The stream, however, embeds the class name, the field names, and the field types, so it is larger than necessary,

reveals the structure of the class to anyone who captures it, and can be consumed only by a Java Virtual Machine that holds a compatible version of the class. Members declared `transient` or `static` are not serialized.

Google Protocol Buffers

Protocol Buffers (Protobuf) [3] is Google's language-neutral and platform-neutral mechanism for serializing structured data. The developer describes each message type once, in a `.proto` file; the `protoc` compiler then generates classes (*stubs*) for the target language, including builders, accessors, and the serialization and parsing code. On the wire, each field is identified by a unique numeric tag instead of its name, and values are encoded compactly, so messages are very small and fast to parse. Because the schema is explicit and shared, mismatches between teams are caught when code is generated and compiled rather than at run time in production.

2.3 Comparative Overview

Table 1 gives the reference comparison provided with the laboratory. It is used throughout the report as the baseline against which the experimental observations are interpreted.

Table 1. Reference comparison of the three serialization techniques.

Criterion	JSON	Java Object Serialization	Protocol Buffers
Format nature	Plain text	Binary byte stream	Compact binary
Human readability	Very high; easy to inspect and debug	Low; hard to inspect without tools, although class metadata is partly visible	None; binary, requires the schema and tools
Language independence	Fully independent; supported by all languages and browsers	Tightly bound to the Java environment (JVM)	Independent of language: C++, Java, Go, Python, and others are supported
Schema	None by default; prone to type-mismatch errors	The Java class itself acts as the schema	Defined precisely in the .proto file
Size and performance	Large; slower parsing	Medium to large; includes class metadata	Very small; very high performance
Exposure in transit	Exposed as plain text	Medium; the class structure inside the message makes it easy to reverse-engineer	Lower; only field numbers and values are sent, without field names

> SCOPE OF THE SECURITY ROW
>
> None of the three formats provides encryption or authentication. The last row of Table 1 concerns how much structural information a captured message reveals; confidentiality and integrity in transit must be provided separately, for example by TLS.

3. Application Scenario and Experimental Environment

3.1 The CustomerPurchaseProfile Scenario

An online store wants to analyze its customers' past purchases by city in preparation for the holiday season, in order to determine suitable pricing and to propose promotional offers. To do this, a complex data object representing a customer's purchase profile (`CustomerPurchaseProfile`) must be transferred between the nodes of the distributed system. Table 2 lists the information the object contains.

Table 2. Attributes of the CustomerPurchaseProfile object.

Attribute	Data type	Sent over the network	Purpose
Customer name	String	Yes	Identifies the customer
City	String	Yes	Allows purchases to be analyzed by city
Purchased item names	List of strings	Yes	Records which products were bought
Item prices	Array of numbers	Yes	Records the price of each product
Holiday-deal eligibility	Boolean	Yes	Indicates whether the customer qualifies for holiday offers
Temporary discount session token	String (memory only)	No	Used locally to compute the discount; must never be sent over the network

The last attribute is included deliberately as a test of control. A well-designed serialization mechanism must allow selected data to remain in memory only, and Experiment 2 verifies this behavior for Java's `transient` keyword.

3.2 Experimental Environment

All experiments were carried out on a single Windows workstation. The components used are listed in Table 3.

Table 3. Software environment used for the experiments.

Component	Details
Operating system	Microsoft Windows (desktop workstation)
Java development	IntelliJ IDEA 2026.2.3 with OpenJDK 27
Build tool	Apache Maven (Experiment 3)
Protocol Buffers	`protoc` compiler and the `protobuf-java` 3.25.5 runtime library
HTTP test server	Python with Flask, listening on port 8080
HTTP client	`curl`, run from the Windows Command Prompt

All distributed “nodes” were simulated on this one machine. The HTTP exchanges used the local loopback address (`localhost`), and the Java and Protocol Buffers experiments exchanged byte arrays in memory or through a file. This does not affect the validity of the observations, because the behavior of a serialization format (what it includes, what it omits, how large it is, and how it fails) does not depend on the physical network.

4. Experiment 1: JSON Serialization and the Lack of a Schema

4.1 Objective and Rationale

This experiment examines JSON, the most widely used text format for web APIs. It has two goals: (a) to represent a complex object in JSON and transmit it with an HTTP request, and (b) to demonstrate, by deliberately introducing a mismatch between two services, the problem created by the *lack of a schema*. JSON is a natural starting point because it makes the data completely visible, and that visibility makes the absence of type information easy to demonstrate.

4.2 Task 1.1: Building and Testing the JSON Object

Procedure

A file named `customer_request.json` was created with an ordinary text editor. It represents the customer and combines several kinds of data: strings, an array of objects, an array of strings, an array of integers, and a boolean value.

customer_request.json JSON

```
{
  "customerName": "Ahmad Saleh",
  "city": "Damascus",
  "pastOrders": [
    { "orderId": 101, "storeBranch": "Main" },
    { "orderId": 205, "storeBranch": "West" }
  ],
  "purchasedItems": ["Laptop", "Wireless Mouse", "Mechanical Keyboard"],
  "itemPrices": [1200, 25, 85],
  "isHolidayDealEligible": true
}
```

The structure can be read as follows:

- `customerName` and `city` are JSON strings.
- `pastOrders` is an array of objects; each object has a numeric `orderId` and a string `storeBranch`.
- `purchasedItems` is an array of strings, and `itemPrices` is an array of integers.
- `isHolidayDealEligible` is a boolean.

Because JSON is plain text, the file can be read and modified directly in any text editor and sent to a server with a general-purpose tool, without any dedicated program.

Test server

The laboratory command posts the file to `http://localhost:8080/api/analyze`. No such service existed on the test machine, so a minimal Flask service was written to receive the request, add up the

item prices, and report the total. It deliberately assumes that `itemPrices` contains *numbers*, which is what makes the mismatch in Task 1.2 observable.

server.py PYTHON

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/api/analyze", methods=["POST"])
def analyze():
    data = request.get_json()
    try:
        # The service expects NUMBERS in itemPrices
        total = sum(data["itemPrices"])
        return jsonify(customer=data["customerName"], total=total)
    except Exception as e:
        return jsonify(error=type(e).__name__, detail=str(e)), 400

if __name__ == "__main__":
    app.run(port=8080)
```

Sending the request

With the server running, the file was sent using `curl` from a second Command Prompt window:

Command Prompt CMD

```
curl -X POST http://localhost:8080/api/analyze -H "Content-Type: application/json" -d
@customer_request.json
```

Result and analysis

Figure 2 shows the response. The server accepted the request and returned the customer's name together with a total of 1310, which is the sum of the three prices in the file (1200 + 25 + 85). The exchange confirms that the JSON object was transmitted correctly and interpreted by the receiver as intended.

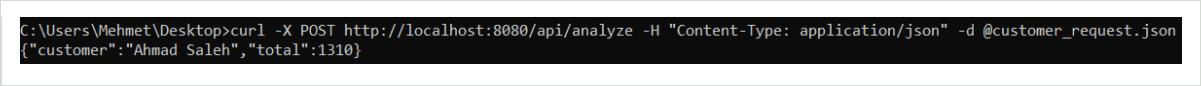


Figure 2. Successful request: the server returns the customer name and the total price (1310).

4.3 Task 1.2: Simulating the Absence of a Single Source of Truth

Scenario

Imagine that two engineers develop two microservices independently, both of which handle item prices. Table 4 summarizes the representation each service assumes.

Table 4. The two incompatible representations of the prices field.

Service	Role	Representation of prices
Service 1	Sender	An array of strings that represent whole numbers: `{"prices": ["1200", "25", "85"]}`
Service 2	Receiver	An array of decimal numbers with two digits of precision: `{"prices": [1200.00, 25.00, 85.00]}`

Reproducing the failure

To reproduce the situation against the test server, a second file, `bad_request.json`, was created. It has the same structure as the first request, but the values of `itemPrices` are strings, exactly as Service 1 would send them:

bad_request.json JSON

```
{
  "customerName": "Test",
  "itemPrices": ["1200", "25", "85"]
}
```

The file was sent with the same command as before, changing only the file name (`-d @bad_request.json`). Figure 3 shows the outcome.

```
C:\Users\Mehmet\Desktop>curl -X POST http://localhost:8080/api/analyze -H "Content-Type: application/json" -d @bad_request.json
{"detail":"unsupported operand type(s) for +: 'int' and 'str'","error":"TypeError"}
```

Figure 3. Type-mismatch failure: the receiver raises a TypeError when prices arrive as strings.

Analysis

The server answered with the error `unsupported operand type(s) for +: 'int' and 'str'`, classified as a `TypeError`. Python's `sum()` begins with the integer 0 and tries to add the first element of the list; because that element is the string `"1200"`, the addition is undefined and the language raises an exception.

Two observations are essential. First, both requests are perfectly valid JSON, and both were accepted by the HTTP layer; the failure occurred only at run time, inside the receiver's business logic. Second, nothing in the message, and nothing in the JSON format itself, states what type `itemPrices` is supposed to have. The error is therefore a symptom of a missing agreement between the two sides, not of a malformed message.

4.4 Report Question 1

> **QUESTION**
>
> When a `DeserializationException` occurs at run time, why can the on-call maintenance engineer not tell which of the two services is correct merely by looking at the JSON message? And how does the absence of a schema affect large distributed systems?

Why the message alone cannot reveal the correct service

A JSON message contains values but no declaration of intent. The arrays `["1200", "25", "85"]` and `[1200.00, 25.00, 85.00]` are both well-formed, and JSON defines no mechanism by which a message can state that `prices` must be strings, integers, or decimals. There is consequently no authoritative contract against which the engineer can compare the message. Looking at the payload shows only *what was sent*, never *what should have been sent*. To find out which service deviates from the intended design, the engineer must search the source code and documentation of both services, and may even have to ask the two development teams, because each of them can legitimately claim that its own representation is the correct one.

Consequences for large distributed systems

The same weakness grows with the size of the system:

- **Late detection.** Type mismatches surface only at run time, in the receiving service, often far from the code that caused them, instead of at compile or build time.
- **Slow and costly diagnosis.** In a landscape of many microservices owned by different teams, locating the team that broke the agreement can take far longer than fixing the defect itself.
- **Silent contract drift.** Without a single shared definition, services can change their data formats independently, and the resulting incompatibilities may remain unnoticed until they reach production.
- **Reliance on informal agreements.** Wikis, emails, and tribal knowledge replace an enforceable contract, and they become outdated as the system evolves.

> **REMEDY**
>
> Define the contract once, in a machine-checkable form (a Protocol Buffers `.proto` file, JSON Schema, or OpenAPI) and validate every message against it. Experiment 3 demonstrates this approach.

4.5 Summary of Findings

- JSON is readable and easy to edit and send, which makes it ideal for exploration and web interfaces.
- The same structure was accepted (Figure 2) or rejected (Figure 3) depending only on the *type* of its values, and JSON gave no way to know which was correct.
- Without a schema, design disagreements between teams become run-time failures.

5. Experiment 2: Standard Java Serialization

5.1 Objective and Rationale

Java provides a built-in serialization mechanism that requires almost no effort from the developer. This experiment studies it from three angles: how a class is prepared for serialization, which parts of an object are deliberately left out (`transient` and `static` members), and what the resulting binary message reveals. The experiment also tests what happens when an object contains a member whose class is *not* serializable. Together these observations explain both the appeal and the limitations of the technique.

5.2 Task 2.1: Writing the Serializable Class

A new Java project was created in IntelliJ IDEA, and the class `CustomerProfile` was added. It carries all the attributes of the scenario in Section 3 and uses the keywords that control what is serialized.

CustomerProfile.java JAVA

```
import java.io.*;
import java.util.Arrays;
import java.util.List;

public class CustomerProfile implements Serializable {
    private static final long serialVersionUID = 1L;

    // Class-level field: NOT serialized
    public static String systemVersion = "v2.4";

    // Temporary field: NOT serialized
    private transient String tempDiscountToken;

    // Regular instance fields: serialized automatically
    private String customerName;
    private List<String> purchasedItems;
    private int[] itemPrices;
    private boolean isHolidayDealEligible;

    // No-argument constructor
    public CustomerProfile() {}

    public CustomerProfile(String customerName, List<String> purchasedItems,
                           int[] itemPrices, boolean isHolidayDealEligible,
                           String token) {
        this.customerName = customerName;
        this.purchasedItems = purchasedItems;
        this.itemPrices = itemPrices;
        this.isHolidayDealEligible = isHolidayDealEligible;
        this.tempDiscountToken = token;
    }
```

```
    @Override
    public String toString() {
        return "CustomerProfile{" +
                "systemVersion='" + systemVersion + '\'' +
                ", tempDiscountToken='" + tempDiscountToken + '\'' +
                ", customerName='" + customerName + '\'' +
                ", purchasedItems=" + purchasedItems +
                ", itemPrices=" + Arrays.toString(itemPrices) +
                ", isHolidayDealEligible=" + isHolidayDealEligible +
                '}';
    }
}
```

The role of each element of the class is explained in Table 5.

Table 5. Elements of the CustomerProfile class and their effect on serialization.

Element	Role in serialization
`implements Serializable`	An empty marker interface. It tells the Java runtime that instances of the class may be converted to a byte stream. Without it, serialization fails.
`serialVersionUID`	A version identifier stored in the stream. On deserialization the runtime compares it with the value in the local class; if they differ, an `InvalidClassException` is thrown, which protects against reading data produced by an incompatible class version.
`static systemVersion`	A class-level field shared by all instances. It belongs to the class and not to any object, so it is **not** serialized.
`transient tempDiscountToken`	An instance field explicitly excluded from serialization. It holds the temporary discount token that must never leave memory.
`customerName`, `purchasedItems`, `itemPrices`, `isHolidayDealEligible`	Ordinary instance fields. They are serialized automatically, provided their own types are serializable (`String`, `List` implementations, `int[]`, and `boolean` all are).
No-argument constructor	Included as the laboratory handout specifies and as is common practice. (Standard Java deserialization restores the field values directly from the stream and does not call a serializable class's own constructors, so this constructor is not what makes restoration work here; it is retained because many frameworks require one.)

5.3 Task 2.2: Performing Serialization and Deserialization

The class `JavaSerializationLab` contains two utility methods and a `main` method. The method `serialize` wraps a `ByteArrayOutputStream` in an `ObjectOutputStream`, writes the object, and returns the resulting `byte[]`. The method `deserialize` performs the reverse operation: it wraps the

byte array in a `ByteArrayInputStream` and an `ObjectInputStream` and calls `readObject()` to rebuild the object.

JavaSerializationLab.java JAVA

```java
import java.io.*;
import java.util.Arrays;

public class JavaSerializationLab {

    // Object -> bytes
    public static byte[] serialize(Object obj) throws IOException {
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(bos);
        oos.writeObject(obj);
        oos.flush();
        return bos.toByteArray();
    }

    // bytes -> object
    public static Object deserialize(byte[] data)
            throws IOException, ClassNotFoundException {
        ObjectInputStream ois =
                new ObjectInputStream(new ByteArrayInputStream(data));
        return ois.readObject();
    }

    public static void main(String[] args) throws Exception {
        CustomerProfile original = new CustomerProfile(
                "Ahmad Saleh",
                Arrays.asList("Laptop", "Mouse"),
                new int[]{1200, 25},
                true,
                "SECRET-TOKEN-999");

        System.out.println("BEFORE serialization:\n" + original);

        byte[] payload = serialize(original);
        System.out.println("\nPayload size: " + payload.length + " bytes");

        // View the raw bytes as text
        System.out.println("\nPayload as text:\n" + new String(payload));

        // Change the static field BEFORE deserializing
        CustomerProfile.systemVersion = "v3.0-UPDATED";

        CustomerProfile reconstructed =
                (CustomerProfile) deserialize(payload);
        System.out.println("\nAFTER deserialization:\n" + reconstructed);
    }
}
```

The `main` method builds a `CustomerProfile`, serializes it, prints the size of the payload and its raw content as text, then **changes the static field `systemVersion` before deserializing**. This last step is intentional: it makes the behavior of static members observable, as discussed next.

5.4 Task 2.3: Results and Observations

The program was executed in IntelliJ IDEA. The console output of this run is too long to be captured in a single screenshot, because it includes the long JVM launch command and the binary payload printed

as text. It is therefore reproduced below as plain text. The unreadable symbols in the “Payload as text” section are expected: they are raw binary bytes that the console tries to display as characters.

Console output TEXT

```
C:\Users\Mehmet\.jdks\openjdk-27\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ
IDEA 2026.2.3\lib\idea_rt.jar=58973" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
Dsun.stderr.encoding=UTF-8 -classpath
C:\Users\Mehmet\Desktop\JavaSerializationLab\out\production\JavaSerializationLab
JavaSerializationLab
BEFORE serialization:
CustomerProfile{systemVersion='v2.4', tempDiscountToken='SECRET-TOKEN-999',
customerName='Ahmad Saleh', purchasedItems=[Laptop, Mouse], itemPrices=[1200, 25],
isHolidayDealEligible=true}

Payload size: 322 bytes

Payload as text:
□□ sr CustomerProfile Z isHolidayDealEligibleL customerNamet Ljava/lang/String;[
itemPricest [IL purchasedItemst Ljava/util/List;xp t Ahmad Salehur [IM□`&v겥 xp □ sr
java.util.Arrays$ArrayList㸿<□□ [ at [Ljava/lang/Object;xpur [Ljava.lang.String;□□V□□ {G xp t
Laptopt Mouse

AFTER deserialization:
CustomerProfile{systemVersion='v3.0-UPDATED', tempDiscountToken='null', customerName='Ahmad
Saleh', purchasedItems=[Laptop, Mouse], itemPrices=[1200, 25], isHolidayDealEligible=true}
```

5.4.1 State of the fields before and after deserialization

Comparing the two `CustomerProfile{...}` lines in the output gives the values in Table 6.

Table 6. Field values before serialization and after deserialization.

Field	Kind	Before	After	Reason
`tempDiscountToken`	transient	SECRET-TOKEN-999	null	Excluded from the stream
`systemVersion`	static	v2.4	v3.0-UPDATED	Not part of the object; read from the class
`customerName`	instance	Ahmad Saleh	Ahmad Saleh	Serialized
`purchasedItems`	instance	[Laptop, Mouse]	[Laptop, Mouse]	Serialized
`itemPrices`	instance	[1200, 25]	[1200, 25]	Serialized
`isHolidayDealEligible`	instance	true	true	Serialized

The transient field. The field `tempDiscountToken` was `SECRET-TOKEN-999` before serialization and `null` afterwards. A `transient` field is skipped when the object is written. When the receiver rebuilds the object, a field with no stored value receives the default for its type: `null` for object references, `0` for numbers, and `false` for booleans. This is exactly the behavior the scenario requires, since the token must remain in memory only. The payload text confirms it: the string `SECRET-TOKEN-999` does not appear anywhere in it.

The static field. The field `systemVersion` shows `v3.0-UPDATED` after deserialization, which at first sight looks like a value that "travelled". It did not. A `static` field belongs to the class itself, of which there is a single copy, and not to any individual object; consequently it is not written to the byte stream at all. The sequence of events was: the object was created while the class held `v2.4`; the object was serialized (without the static field); the program then changed the class's single copy to `v3.0-UPDATED`; finally the new object was rebuilt and, when printed, simply read the class's *current* value.

> **KEY POINT**
>
> Static values do not travel with serialized objects. In a different JVM, the restored object would show whatever value that JVM's copy of the class holds, for example `v2.4` in a freshly started process.

5.4.2 Serializing an object that contains a non-serializable member

To observe how the mechanism reacts to a member whose class is not serializable, a class `Address` was created without the `Serializable` interface and added as a field of `CustomerProfile`:

Address.java JAVA

```
public class Address {          // does NOT implement Serializable
    String street = "Main St";
}
```

CustomerProfile.java (excerpt) JAVA

```
public class CustomerProfile implements Serializable {
    // ... existing fields ...
    private Address address = new Address();   // added for this test
    // ...
}
```

Running the program again produced the failure shown in Figure 4. The "BEFORE serialization" line was still printed, because the failure occurs later, when `writeObject` is called.

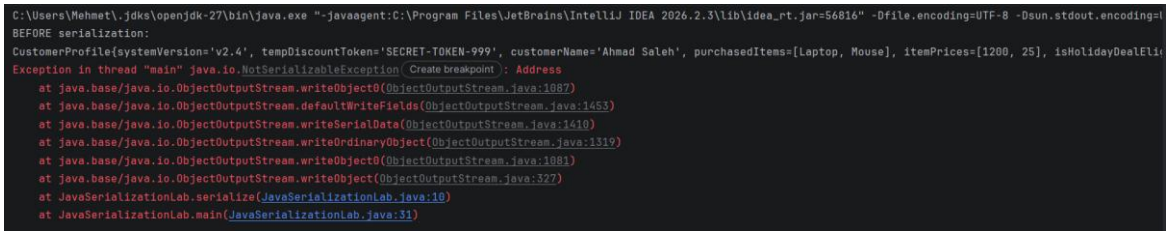


Figure 4. The program fails with java.io.NotSerializableException: Address.

The screenshot is wide, so the exception and the stack trace it shows are transcribed below for easier reading.

Transcription of Figure 4 TEXT

```
Exception in thread "main" java.io.NotSerializableException: Address
    at java.base/java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1087)
    at java.base/java.io.ObjectOutputStream.defaultWriteFields(ObjectOutputStream.java:1453)
    at java.base/java.io.ObjectOutputStream.writeSerialData(ObjectOutputStream.java:1410)
    at java.base/java.io.ObjectOutputStream.writeOrdinaryObject(ObjectOutputStream.java:1319)
    at java.base/java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1081)
    at java.base/java.io.ObjectOutputStream.writeObject(ObjectOutputStream.java:327)
    at JavaSerializationLab.serialize(JavaSerializationLab.java:10)
    at JavaSerializationLab.main(JavaSerializationLab.java:31)
```

The exception recorded is **java.io.NotSerializableException: Address**. Reading the stack trace from the bottom upwards shows the path: `main` (line 31 of `JavaSerializationLab.java`) called `serialize` (line 10), which called `ObjectOutputStream.writeObject`; the runtime then walked through the fields of `CustomerProfile` (`defaultWriteFields`) and reached the `address` field, whose class does not implement `Serializable`.

This illustrates the rule that governs the technique: Java serializes the entire *object graph* reachable from the object being written, so every object in that graph must be serializable. There are two remedies: make `Address` implement `Serializable`, or mark the `address` field `transient` if its data should not be transmitted (in which case it will be `null` after deserialization). After this test the extra field was removed so that the rest of the experiment could proceed with the original class.

5.4.3 Inspecting the content of the payload

The last part of the task prints the payload as text. Although most of the bytes are unreadable, readable fragments stand out, and they are summarized in Table 7.

Table 7. Readable fragments found in the Java-serialized payload.

Fragment in the payload	What it reveals
`¬í sr` at the start	The stream header (bytes `AC ED 00 05`, shown as unprintable symbols) followed by `s` and `r`, the markers (0x73 and 0x72) that introduce an object and its class description.
`CustomerProfile`	The fully qualified name of the serialized class.
`isHolidayDealEligible`, `customerName`, `itemPrices`, `purchasedItems`	The names of the serialized fields.
`Ljava/lang/String;`, `[I`, `Ljava/util/List;`	The type signatures of the fields (a `String`, an `int` array, and a `List`).
`java.util.Arrays$ArrayList`, `[Ljava.lang.String;`	The concrete classes Java used internally for the list and its backing array, which are implementation details of the sender.

These observations have three consequences. **Size:** the payload is 322 bytes for an object whose actual data (a name, two product names, two prices, and a flag) occupies only a few dozen bytes; the remainder is class metadata. **Exposure:** anyone who intercepts the message learns the structure of the

class and the Java types used, which makes the format easy to reverse-engineer. **Coupling:** the message is meaningful only to a Java environment that has a compatible version of the class. In addition, deserializing data that comes from an untrusted source is a well-documented security risk [4], which is why this technique should not be used to read external input.

5.5 Summary of Findings

- `transient` fields are excluded from the stream and restored with default values, which provides a simple way to keep sensitive or temporary data in memory.
- `static` fields are never serialized, because they belong to the class and not to the object.
- Every object in the graph must be serializable; otherwise a `NotSerializableException` is raised at the moment of writing.
- The payload (322 bytes) embeds the class name, field names, and field types, which makes it large, easy to inspect for an eavesdropper, and tied to Java.

6. Experiment 3: Google Protocol Buffers

6.1 Objective and Rationale

The previous experiments exposed two weaknesses: JSON has no enforceable contract, and Java serialization is bulky, exposes class metadata, and is tied to the JVM. Protocol Buffers address both. In this experiment the `CustomerProfile` message is defined in a schema file, a Java class is generated from it, the object is serialized, the size of the result is measured, and the way the binary format identifies fields is tested by renaming a field on the receiving side only.

6.2 Preparing the Environment

Two components are needed: the `protoc` compiler, which translates `.proto` files into Java source code, and the `protobuf-java` runtime library, which the generated code depends on. The runtime was added to a new Maven project through the following dependency in `pom.xml`, after which Maven's "Reload" action downloaded it.

pom.xml (excerpt) XML

```xml
<dependencies>
    <dependency>
        <groupId>com.google.protobuf</groupId>
        <artifactId>protobuf-java</artifactId>
        <version>3.25.5</version>
    </dependency>
</dependencies>
```

> **VERSION ALIGNMENT**
>
> The version of the `protoc` compiler and the version of the `protobuf-java` runtime must correspond. A mismatch is the most common cause of build errors in this experiment (see Appendix A.3).

6.3 Task 3.1: Defining the Schema and Generating the Stub

The file `customer_profile.proto` was created in the project's root folder. It describes the message structure using field rules and unique numeric tags.

customer_profile.proto PROTOBUF

```
syntax = "proto2";
package distributed.lab;
option java_package = "com.distributed.lab";
option java_outer_classname = "CustomerProfileProto";

message CustomerProfile {
  required string customer_name = 1;
  optional string city = 2 [default = "Damascus"];
  repeated string purchased_items = 3;
  repeated int32 item_prices = 4;
  optional bool is_holiday_deal_eligible = 5 [default = false];
}
```

Each field declaration consists of a rule, a type, a name, and a tag number. Table 8 summarizes the schema.

Table 8. Fields of the CustomerProfile message.

Field name	Rule	Type	Tag	Default / behavior
`customer_name`	required	string	1	Must always be set; building the message without it fails
`city`	optional	string	2	May be absent; reads as "Damascus" when unset
`purchased_items`	repeated	string	3	A list; empty when no items are added
`item_prices`	repeated	int32	4	A list; empty when no prices are added
`is_holiday_deal_eligible`	optional	bool	5	May be absent; reads as false when unset

The rules have the following meaning. A **required** field must be present in every message. An **optional** field may be omitted; if it is, the reader sees the declared default value and can use a generated `has…()` method to tell whether the field was actually set. A **repeated** field may occur any number of times and represents a list. The **numeric tag** is the field's permanent identity in the binary format; the field name is only a label used in the schema and in the generated code. (Modern Protobuf guidance discourages `required` because a required field can never be removed without breaking compatibility; it is used here because the laboratory specifies the proto2 syntax.)

The Java class was then generated with the compiler:

Command Prompt CMD

```
protoc --java_out=src/main/java customer_profile.proto
```

The command creates the file `src/main/java/com/distributed/lab/CustomerProfileProto.java`. Inspecting it shows the methods the compiler wrote automatically: a `Builder` obtained with `newBuilder()`; a setter and getter for each field (`setCustomerName`, `getCustomerName`, and so on); `addAll…` methods for the repeated fields; `hasCity()` for the optional field; and the serialization and parsing methods `toByteArray()` and `parseFrom(...)`. None of this code is written by hand, which is why the sender and receiver cannot disagree about the structure of the message as long as they use the same schema.

6.4 Task 3.2: Building the Object, Serializing It, and Measuring the Result

The program `ProtobufLab` builds the object with the Builder pattern, serializes it with `toByteArray()`, prints the size, restores the object with `parseFrom(...)`, and reads a few fields back. For the later

comparison it also prints the size of the equivalent compact JSON text, and it saves the payload to a file named `payload.bin` that the receiver will use in Section 6.6.

ProtobufLab.java JAVA

```java
import com.distributed.lab.CustomerProfileProto.CustomerProfile;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Arrays;

public class ProtobufLab {
    public static void main(String[] args) throws Exception {
        // Build the object with the Builder pattern
        CustomerProfile profile = CustomerProfile.newBuilder()
                .setCustomerName("Ahmad Saleh")
                .addAllPurchasedItems(Arrays.asList("Laptop", "Mouse"))
                .addAllItemPrices(Arrays.asList(1200, 25))
                .setIsHolidayDealEligible(true)
                .build();

        // Serialize to bytes
        byte[] protoPayload = profile.toByteArray();
        System.out.println("Protobuf size: " + protoPayload.length + " bytes");

        // Save the bytes to a file (used later by the receiver)
        Files.write(Path.of("payload.bin"), protoPayload);

        // Deserialize
        CustomerProfile restored = CustomerProfile.parseFrom(protoPayload);
        System.out.println("Name: " + restored.getCustomerName());
        System.out.println("City: " + restored.getCity());
        System.out.println("hasCity: " + restored.hasCity());

        // JSON size of the same data (compact), for the comparison
        String json = "{\"customerName\":\"Ahmad Saleh\","
                + "\"purchasedItems\":[\"Laptop\",\"Mouse\"],"
                + "\"itemPrices\":[1200,25],"
                + "\"isHolidayDealEligible\":true}";
        System.out.println("JSON size: " + json.getBytes("UTF-8").length
                + " bytes");
    }
}
```

Figure 5 shows the output.

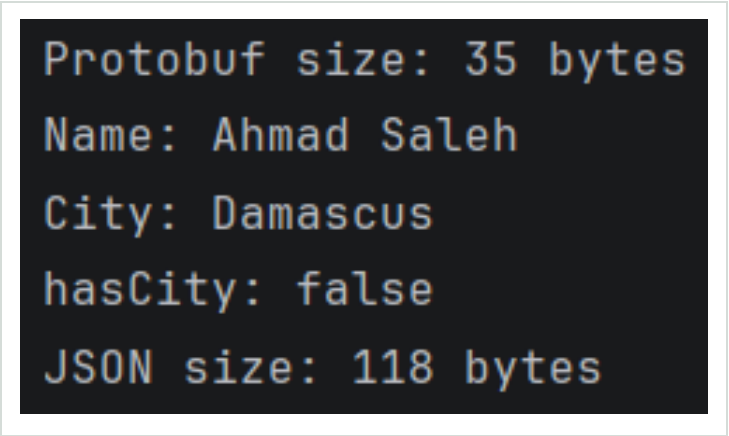


Figure 5. Output of ProtobufLab: size, restored values, and the default city.

The output is interpreted line by line:

- **Protobuf size: 35 bytes.** This is the size of the complete serialized message.
- **Name: Ahmad Saleh.** The object was restored correctly from the bytes.
- **City: Damascus** and **hasCity: false.** The `city` field was never set, so it was *not transmitted at all*. The reader nevertheless obtains the schema's default value, `Damascus`, from `getCity()`, while `hasCity()` correctly reports that the field was absent from the message.
- **JSON size: 118 bytes.** The compact JSON form of the same data, printed for the comparison in Section 6.5.

Why the message is so small

Each field is encoded as a one-byte **tag** (the field number combined with a wire type) followed by its value; strings additionally carry a one-byte length. Field names, type names, and class metadata are never sent. Figure 6 breaks the 35 bytes down, and Table 9 explains every group. The tag byte is computed as `(field number << 3) | wire type`. For field 1 (a string, wire type 2) this gives `(1 << 3) | 2 = 0x0A`; for field 4 (an integer, wire type 0) it gives `(4 << 3) | 0 = 0x20`.

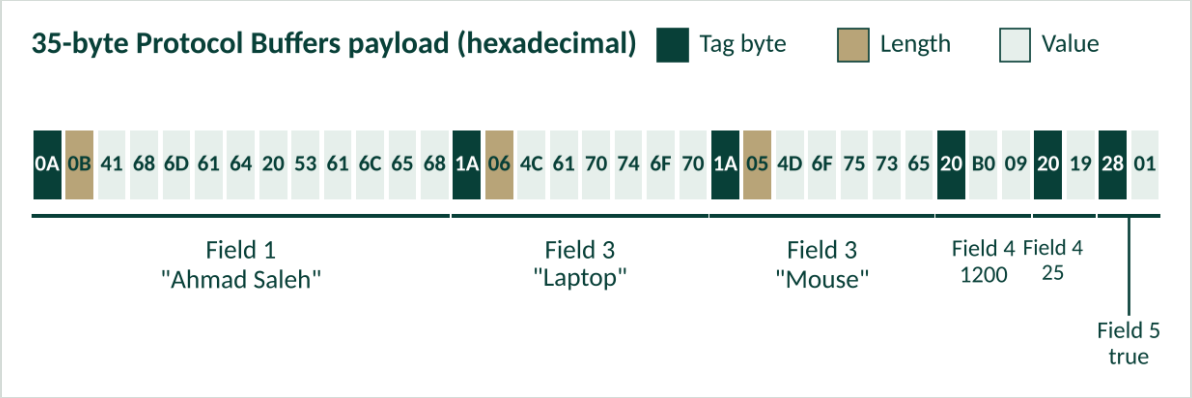


Figure 6. Byte-level layout of the 35-byte Protocol Buffers payload.

Table 9. Decomposition of the 35-byte payload.

Bytes (hexadecimal)	Field	Meaning	Size
`0A 0B` + `41 68 6D 61 64 20 53 61 6C 65 68`	1	Tag, length 11, and the text "Ahmad Saleh"	13
`1A 06` + `4C 61 70 74 6F 70`	3	Tag, length 6, and the text "Laptop"	8
`1A 05` + `4D 6F 75 73 65`	3	Tag, length 5, and the text "Mouse"	7
`20 B0 09`	4	Tag and the integer 1200 as a two-byte variable-length value	3
`20 19`	4	Tag and the integer 25 (`0x19`)	2
`28 01`	5	Tag and the boolean true (1)	2
Total		Field 2 (`city`) is absent and costs zero bytes	**35**

Two details deserve attention. First, the integer 1200 occupies only two bytes (`B0 09`) thanks to Protobuf's variable-length integer encoding, which uses fewer bytes for smaller numbers. Second, each element of the repeated field `item_prices` carries its own tag (`20`), because in proto2 repeated numeric fields are not packed by default.

6.5 Report Question 2 (a): Comparing the Message Sizes

> **QUESTION**
>
> Compare the size of the byte array produced by Protocol Buffers with its size in Java serialization and in JSON for exactly the same data.

The same data was serialized with each technique: the customer name, two product names, two prices, and the eligibility flag. The JSON text was measured in its compact form (no extra whitespace) and encoded as UTF-8; the Java figure is the measured payload of Experiment 2, in which the `transient` token is excluded; the Protobuf figure is the length of `toByteArray()`. The results are given in Table 10.

Table 10. Serialized size of identical data in each format.

Technique	Size (bytes)	Relative to Protobuf	What occupies the space
Protocol Buffers	35	1.0×	Tags, length prefixes, and values only
JSON (compact)	118	3.4×	Every field name as quoted text, plus quotes, colons, commas, and brackets
Java Serialization	322	9.2×	Class description, field names, type signatures, and the implementation classes of the list and arrays

Protocol Buffers produced the smallest message by a wide margin: about 70% smaller than JSON and about 89% smaller than Java serialization. The reason is structural. Protobuf transmits a tag and a value for each field and nothing else. JSON must repeat the name of every field as text in every message; and Java serialization embeds a full description of the class, its fields, and their types in the stream. (Figure 8 in Section 7 presents the same figures graphically.)

> **INTERPRETING THE NUMBERS**
>
> These figures apply to this small object. For messages dominated by long text values, the difference narrows because the text content, not the field overhead, accounts for most of the size. For small, field-heavy messages such as this one, the advantage of Protocol Buffers is at its largest.

6.6 Report Question 2 (b): Renaming a Field on the Receiver Only

> **QUESTION**
>
> Change the field name `customer_name = 1` in the `.proto` file to `full_client_name = 1`, and regenerate the class on the receiver's side only, without changing the sender. Does deserialization succeed? Explain, based on how the numeric tags work.

Test design

The sender must remain untouched, so the original schema and the original generated class were kept. For the receiver, a separate schema file, `customer_profile_receiver.proto`, was created. It is identical to the original except that field 1 is renamed `full_client_name` and that it uses a different package and outer class name so that both versions can coexist in the same project:

customer_profile_receiver.proto PROTOBUF

```
syntax = "proto2";
package distributed.lab.receiver;
option java_package = "com.distributed.lab.receiver";
option java_outer_classname = "CustomerProfileReceiverProto";

message CustomerProfile {
  required string full_client_name = 1;   // renamed from customer_name
  optional string city = 2 [default = "Damascus"];
  repeated string purchased_items = 3;
  repeated int32 item_prices = 4;
  optional bool is_holiday_deal_eligible = 5 [default = false];
}
```

The receiver class was generated with the same command as before, applied to the new file (`protoc --java_out=src/main/java customer_profile_receiver.proto`). The receiver program then reads `payload.bin`, which had been written earlier by the *original* sender, prints the raw bytes in hexadecimal, and parses them with the renamed schema:

ProtobufReceiver.java JAVA

```
import com.distributed.lab.receiver.CustomerProfileReceiverProto.CustomerProfile;
import java.nio.file.Files;
import java.nio.file.Path;

public class ProtobufReceiver {
    public static void main(String[] args) throws Exception {
        // Bytes produced by the ORIGINAL sender (ProtobufLab), unchanged
        byte[] payload = Files.readAllBytes(Path.of("payload.bin"));
```

```
        // Show the raw bytes in hexadecimal
        StringBuilder hex = new StringBuilder();
        for (byte b : payload) hex.append(String.format("%02X ", b));
        System.out.println("Payload (hex): " + hex);

        // Parse with the receiver's renamed schema
        CustomerProfile received = CustomerProfile.parseFrom(payload);

        System.out.println("Full client name: " + received.getFullClientName());
        System.out.println("City: " + received.getCity());
        System.out.println("Items: " + received.getPurchasedItemsList());
        System.out.println("Prices: " + received.getItemPricesList());
        System.out.println("Holiday eligible: "
                + received.getIsHolidayDealEligible());
    }
}
```

Result

Figure 7 shows the outcome. The first line is the hexadecimal content of the payload; it is cut at the right edge of the screenshot, and the complete byte sequence is listed in Table 9. The three `WARNING` lines are a harmless message from the JVM and are explained in Appendix A.2.

```
Payload (hex): 0A 0B 41 68 6D 61 64 20 53 61 6C 65 68 1A 06 4C 61 70 74 6F 70 1A 05 4D 6F 75 73 65 20 B0 09 20 19
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::arrayBaseOffset has been called by com.google.protobuf.UnsafeUtil$MemoryAccessor (file:/
WARNING: Please consider reporting this to the maintainers of class com.google.protobuf.UnsafeUtil$MemoryAccessor
WARNING: sun.misc.Unsafe::arrayBaseOffset will be removed in a future release
Full client name: Ahmad Saleh
City: Damascus
Items: [Laptop, Mouse]
Prices: [1200, 25]
Holiday eligible: true
```

Figure 7. The receiver parses the unchanged payload using the renamed field.

Deserialization succeeds. The receiver, which knows the field only as `full_client_name`, correctly obtained the value “Ahmad Saleh”, and every other field was also read without error.

Explanation

The explanation lies in the first bytes of the payload, `0A 0B 41 68 6D …`. The byte `0A` encodes the **field number 1** and the wire type “length-delimited”; `0B` is the length (11 bytes); the following bytes are the text “Ahmad Saleh”. The words `customer_name` appear nowhere in the message. The message states only that “field 1 has this value”. When the receiver parses the bytes, it reads tag 1 and looks up which field its *own* schema places at number 1; in the receiver’s schema that field is called `full_client_name`, so the value is stored there. The name exists only in the `.proto` file and in the generated code, as a convenience for humans and programs; it has no role on the wire.

The rename therefore has exactly one consequence, and it concerns source code rather than data: the generated accessor changes from `getCustomerName()` to `getFullClientName()`, so any receiver code that used the old method must be updated. In JSON, by contrast, a field is identified by its name, so renaming a key on one side only would leave the other side unable to find the value.

COROLLARY

If the *number* had been changed instead (for example `full_client_name = 9`), the receiver would no longer recognize the data carried under tag 1, and a required-field error would be expected. In Protocol Buffers, names are free to change but numbers are permanent. This is the principle behind discussion question 4 in Section 8.4.

6.7 Summary of Findings

- The same object occupied only 35 bytes in Protocol Buffers, compared with 118 bytes in JSON and 322 bytes in Java serialization.
- Unset optional fields cost nothing on the wire, while the reader still obtains the schema's default value.
- Fields are identified by tag number, not by name, so a field can be renamed on one side without breaking deserialization.
- Because the schema is explicit and the code is generated, both sides share one definition of the message, which removes the type-mismatch problem of Experiment 1.

7. Comparative Analysis of the Three Techniques

7.1 Consolidated Experimental Results

Figure 8 places the three measured message sizes side by side. They concern the same data in every case: the customer name, two product names, two prices, and the eligibility flag.

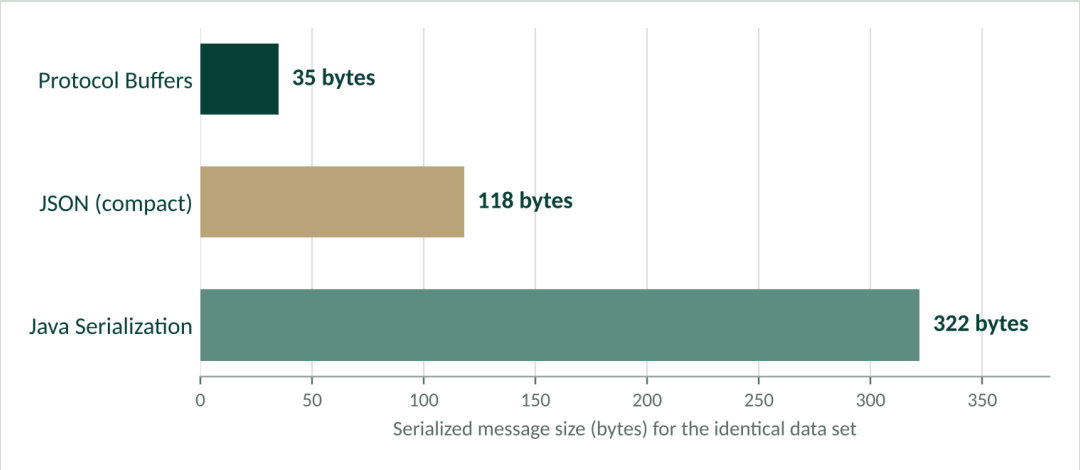


Figure 8. Serialized size of identical data: Protocol Buffers, compact JSON, and Java serialization.

Table 11 combines the size measurements with the other behaviors observed during the three experiments. It complements the reference comparison in Table 1 by showing the evidence obtained in the laboratory.

Table 11. Evidence gathered in the laboratory for each technique.

Criterion	JSON	Java Object Serialization	Protocol Buffers
Measured size (same data)	118 bytes	322 bytes	35 bytes
Contract enforcement	None; the type mismatch surfaced only at run time (Figure 3)	Implicit in the Java class; both ends need a compatible class, and failures such as the one in Figure 4 appear at run time	Explicit `.proto` schema; code is generated from a single definition
Information exposed by the payload	All field names and values in plain text	Class name, field names, types, and internal classes (Table 7)	Tags and values only (Table 9)
Effect of renaming a field on one side	The key is the identifier, so the receiver no longer finds the value	Fields are matched by name, so a renamed field is not matched and keeps its default value	None on the wire; only the generated accessor name changes (Section 6.6)
Language independence	Any language and any browser	Java (JVM) only	Many languages through generated code

Criterion	JSON	Java Object Serialization	Protocol Buffers
Ease of debugging	Excellent; readable with any text editor	Poor; binary with partial metadata visible	Poor without tools; requires the schema
Setup effort	None	Add `implements Serializable`	Write the `.proto` file, install `protoc`, add the runtime library

Only the Protocol Buffers rename was tested directly (Section 6.6). The entries for JSON and Java serialization in that row follow from how each format identifies fields, by key name and by field name respectively.

7.2 Interpretation

The results show that no technique is best on every criterion; each one optimizes a different set of properties. JSON optimizes **openness**: anyone can read it, and every platform can produce and consume it, but it pays for that with larger messages and with the absence of a contract. Java serialization optimizes **convenience** inside a closed Java environment: a single keyword is enough, but the price is language lock-in, exposed internals, and bulky output. Protocol Buffers optimize **efficiency and discipline**: the messages are the smallest, and the schema prevents disagreements between teams, but the developer must accept an additional build step and give up direct readability.

Three questions therefore guide the choice of technique:

1. **Who are the consumers?** If they include browsers or arbitrary third parties, JSON is almost always required; if they are all Java programs of the same version, Java serialization is possible; if they are services in several languages under one organization, Protocol Buffers fit best.
2. **How important are size and speed?** Where bandwidth and latency are critical, the compact binary encoding of Protocol Buffers is decisive.
3. **How strict must the contract be?** Where independent teams must be prevented from disagreeing about the data, an explicit schema is indispensable.

8. Design Decisions: Choosing a Serialization Technique

The closing part of the laboratory asks, as a distributed-systems engineer, for the best technique for four situations, with justification. The answers below apply the evidence from Sections 4 to 7 and are summarized in Table 12.

Table 12. Summary of recommended techniques.

Case	Situation	Recommended	Decisive factors
1	Web shopping-cart data sent from a JavaScript browser client to a back-end server	JSON	Native browser support; language independence; easy to debug
2	Fast-moving Java-only prototype with three nodes on the same JVM version	Java Serialization	No libraries or compilers; one keyword; homogeneous environment
3	Large multi-language trading system (Java, C++, Go) with strict contract and minimal bandwidth	Protocol Buffers	Multi-language code generation; enforced schema; smallest and fastest messages
4	Why a deleted field's tag number must never be reused	Not applicable	The tag is the field's only identity on the wire

8.1 Case 1: Shopping-Cart Data from a Browser to the Back End

> **RECOMMENDED: JSON**
>
> A web application running in the user's browser (written in JavaScript) sends shopping-cart data to a back-end server.

JSON is the appropriate choice, for four reasons:

- **Native support in the browser.** JavaScript offers `JSON.stringify()` and `JSON.parse()` as built-in functions, so no library or build step is required on the client.
- **Language independence.** The back end can be written in any language, and every mainstream language has JSON support.
- **Ease of debugging.** Requests and responses can be read directly in the browser's developer tools.
- **Adequate efficiency.** A shopping cart is small, so the larger size of JSON is not a significant cost.

Java serialization cannot be produced or read by browser JavaScript at all. Protocol Buffers would work but would add tooling and a build step to the front end, and their size advantage is not needed for a message of this kind.

8.2 Case 2: A Fast-Moving Java-Only Prototype

> **RECOMMENDED: JAVA OBJECT SERIALIZATION**
>
> An internal prototype of three nodes, all written in Java on the same JVM version, must share complex objects quickly.

The prototype must share complex objects between its nodes quickly, **without setting up external libraries or additional compilers**. **Java serialization** satisfies this precisely:

- **Zero setup.** A class only needs `implements Serializable`; there are no schema files, compilers, or third-party libraries.
- **Compatible environment.** Because all nodes use Java and the same JVM version, the technique's usual weaknesses (language lock-in and sensitivity to class versions) do not matter.
- **Complex objects are handled automatically.** Lists, arrays, and nested objects are serialized without mapping code.

JSON would require mapping code or an additional library, and Protocol Buffers would require `.proto` files and the `protoc` compiler, which is exactly the extra setup the situation excludes.

> **LIMITATION**
>
> This choice is suitable for a prototype, not for production or for input from untrusted sources. Experiment 2 showed that the format exposes class metadata, and deserializing untrusted Java data is a recognized security risk [4]. If the prototype evolves into a production system, migration to a schema-based format should be planned.

8.3 Case 3: A Large Multi-Language Trading System

> **RECOMMENDED: PROTOCOL BUFFERS**
>
> A large financial trading system made of microservices in several languages needs maximum security, minimum bandwidth, and a strict contract.

The services are written in Java, C++, and Go, and the system requires the highest level of security, the lowest possible bandwidth consumption, and a strict contract that prevents type-mismatch errors between the development teams. **Protocol Buffers** meet each requirement:

- **Multiple languages.** The same `.proto` file generates code for Java, C++, and Go, so every service uses an identical definition. Java serialization cannot cross language boundaries.
- **Strict contract.** The schema is the single source of truth, and mismatches are caught when code is generated and compiled, which eliminates the failure studied in Experiment 1.
- **Minimal bandwidth and fast parsing.** The measurements showed 35 bytes against 118 (JSON) and 322 (Java). In a trading environment, smaller and faster-to-parse messages directly reduce latency and network cost.
- **Reduced exposure.** Only field numbers and values are sent, not field names or class metadata, and Protocol Buffers avoid the deserialization risks of Java object streams.

JSON would be larger, slower to parse, and would not enforce types. Protocol Buffers do not encrypt data, so a secure transport such as TLS is still required to meet the security requirement in full.

8.4 Additional Question: Why a Deleted Field's Tag Must Not Be Reused

> **QUESTION**
>
> Why does Protocol Buffers prohibit reusing the numeric tag of an `optional` field that was deleted when the system is updated?

The prohibition follows directly from what Experiment 3 demonstrated: **the tag number is the field's only identity in the binary message.** The name is not transmitted, so the receiver has nothing but the number with which to decide what a piece of data means. Consider what happens if the field `city = 2` (a string) is deleted in a later version of the schema, and a new field `discount_rate = 2` (an integer) is added:

1. Messages written earlier (stored files, queued messages, logs) still contain tag 2 with the **old meaning**: a city name.
2. Clients and servers that have not yet been updated continue to send tag 2 as a city string.
3. An updated receiver reads tag 2 and interprets it as `discount_rate`. Depending on whether the encodings happen to be compatible, the value is either **misinterpreted** (for example an old integer read as a different quantity) or **silently dropped** as an unknown field.
4. In both cases the system keeps running with incorrect data, and no error points to the cause.

The correct practice is to treat a retired number as permanently unavailable and to tell the compiler so by marking it `reserved`; any later attempt to reuse the number or the old name is then rejected at compile time:

customer_profile.proto (excerpt) PROTOBUF

```
message CustomerProfile {
  reserved 2;              // formerly: optional string city = 2;
  reserved "city";         // the old name is retired as well
  required string customer_name = 1;
  repeated string purchased_items = 3;
  // ... remaining fields ...
}
```

9. Conclusion

9.1 Summary of Results

This laboratory examined how a complex object can be transferred between the nodes of a distributed system using three serialization approaches, and measured their behavior on a single scenario. The main findings are the following.

- **JSON** is open, readable, and universally supported, but it defines no data contract. The type mismatch reproduced in Experiment 1 was valid JSON and failed only at run time, which shows how the absence of a schema turns design disagreements into operational incidents.
- **Java object serialization** is the easiest technique to adopt in a pure Java setting. Experiment 2 showed that `transient` and `static` members are not serialized, that every object in the graph must be serializable, and that the 322-byte payload embeds class metadata that reveals the structure of the class and binds the message to the JVM.
- **Protocol Buffers** produced the smallest message (35 bytes), enforce a shared schema, and identify fields by number, so a field can be renamed on one side without breaking deserialization (Experiment 3).

For identical data, Protocol Buffers were about 70% smaller than JSON and about 89% smaller than Java serialization. Yet the experiments also showed that the right choice depends on context rather than on size alone: openness favors JSON for browser-facing interfaces, convenience favors Java serialization for closed Java prototypes, and discipline together with efficiency favors Protocol Buffers for large, multi-language, performance-critical systems.

9.2 Recommendations

- Define every data contract in a machine-checkable form, and validate messages against it as early as possible.
- Use JSON for browser-facing and public interfaces where openness and debuggability matter most.
- Restrict Java serialization to closed, trusted, same-version Java environments, and never use it to read untrusted input.
- In schema-based systems, treat tag numbers as permanent, mark retired numbers and names as `reserved`, and avoid `required` fields in new designs.
- Remember that none of these formats encrypts data; use a secure transport for sensitive traffic.

Appendix A: Practical Notes and Troubleshooting

This appendix records practical issues met while carrying out the experiments. They are unrelated to the concepts under study but may save time to anyone repeating the work.

A.1 Quoting JSON in the Windows Command Prompt

The Windows Command Prompt does not treat single quotes as quotation marks. A first attempt to send the mismatched request inline, with the JSON enclosed in single quotes after `-d`, therefore delivered a malformed body, and the server replied with its generic HTML “400 Bad Request” page before reaching the application logic. This is a shell-quoting problem and not a problem of the server or of the lab. There are two simple solutions. The first, used in this report, is to place the JSON in a file and pass it with `-d @file`. The second is to enclose the JSON in double quotes and escape the inner quotes with backslashes:

Command Prompt CMD

```
curl -X POST http://localhost:8080/api/analyze -H "Content-Type: application/json" -d
"{\"customerName\":\"Test\",\"itemPrices\":[\"1200\",\"25\",\"85\"]}"
```

A.2 JVM Warnings Printed by the Protobuf Runtime

When the Protocol Buffers programs were run, the console showed several lines beginning with `WARNING: ... sun.misc.Unsafe ...` (see Figure 7). They arise because `protobuf-java` 3.25.5 uses an internal JDK memory-access method that recent JDK releases mark as terminally deprecated. The messages are only warnings; they do not affect the results of the experiments and can be ignored.

A.3 Keeping protoc and the Runtime Library Aligned

Code generated by `protoc` depends on the runtime library, so the two must come from compatible releases. In this work the runtime was `protobuf-java` 3.25.5. When a build fails with errors about missing or incompatible generated classes, the first check should be `protoc --version` against the version declared in `pom.xml`.

A.4 Making Binary Payloads Easier to Read

Printing a payload with `new String(payload)` is the simplest way to look at it, but most of the output is unreadable. Two alternatives are shown below: replacing non-printable bytes with dots, which

makes embedded names easy to spot, and printing each byte as a hexadecimal number, which was used in Experiment 3.

Helper snippets JAVA

```java
// Replace every non-printable byte with a dot
public static String toReadable(byte[] data) {
    StringBuilder sb = new StringBuilder();
    for (byte b : data) {
        sb.append(b >= 32 && b <= 126 ? (char) b : '.');
    }
    return sb.toString();
}

// Print the bytes as hexadecimal numbers
for (byte b : payload) System.out.printf("%02X ", b);
```

References

[1] T. Bray (Ed.), “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Internet Engineering Task Force, December 2017. https://www.rfc-editor.org/rfc/rfc8259

[2] Oracle, “Java Object Serialization Specification,” Java SE documentation. https://docs.oracle.com/en/java/javase/17/docs/specs/serialization/index.html

[3] Google, “Protocol Buffers Documentation” (Language Guide and Encoding). https://protobuf.dev/

[4] OWASP Foundation, “Deserialization Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html

[5] Network Communication Laboratory handout, “Transferring Complex Data Objects in Distributed Systems (Data Serialization and Deserialization).”